

2026

物件導向

第11組:貪食蛇小遊戲

組員:

資工2A胡閔傑，資工2A鍾文迪，資工2A萬紫菱

瞭解更多 ►

Travel Plan

目錄

3	4	5	6	7
遊戲介紹	遊戲說明	操作方式	程式結構說明	安裝與執行
8	9	10	11	
運行畫面截圖	程式畫面截圖	UML圖	分工	

遊戲介紹

本專題使用 C++ 製作一款在 Windows 主控台執行的貪食蛇遊戲。玩家需要控制蛇在地圖中移動並吃掉食物，使蛇身逐漸變長並累積分數。隨著分數提高，蛇的移動速度也會加快。

Travel Plan

遊戲規則

1. 蛇會持續朝目前方向前進。
2. 地圖中的 @ 代表食物，蛇頭碰到食物即視為吃到食物。
3. 每吃到一個食物可獲得 10 分，蛇身會增加一格。
4. 每累積 50 分，蛇的移動速度會加快。
5. 蛇不能直接轉向目前方向的正反方向。
6. 蛇頭撞到牆壁或自己的身體時，遊戲結束。
7. 遊戲結束後可選擇重新開始(R)或離開遊戲(Q)。

操作方式

畫面符號說明：

- O：蛇頭
- o：蛇身
- @：食物
- #：牆壁



W 或方向鍵上

向上移動



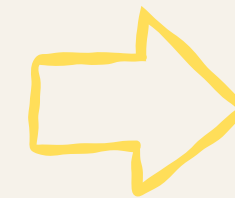
S 或方向鍵下

向下移動



A 或方向鍵左

向左移動



D 或方向鍵右

向右移動



Q

離開遊戲



R

遊戲結束後重新開始



程式結構說明

SnakeGame.h

宣告 Direction、Point、Snake、Food、Game 等類別與結構
定義蛇、食物與遊戲流程所需的成員變數
宣告各類別的成員函式，例如移動、繪圖、判斷碰撞等

main.cpp

程式進入點
建立 Game 物件
呼叫 game.run() 啟動貪食蛇遊戲

SnakeGame.cpp

實作所有類別的方法細節
處理蛇的移動、變長、碰撞判斷
處理食物的隨機生成
處理鍵盤輸入、畫面更新、分數與遊戲結束判斷

安裝與執行

系統需求

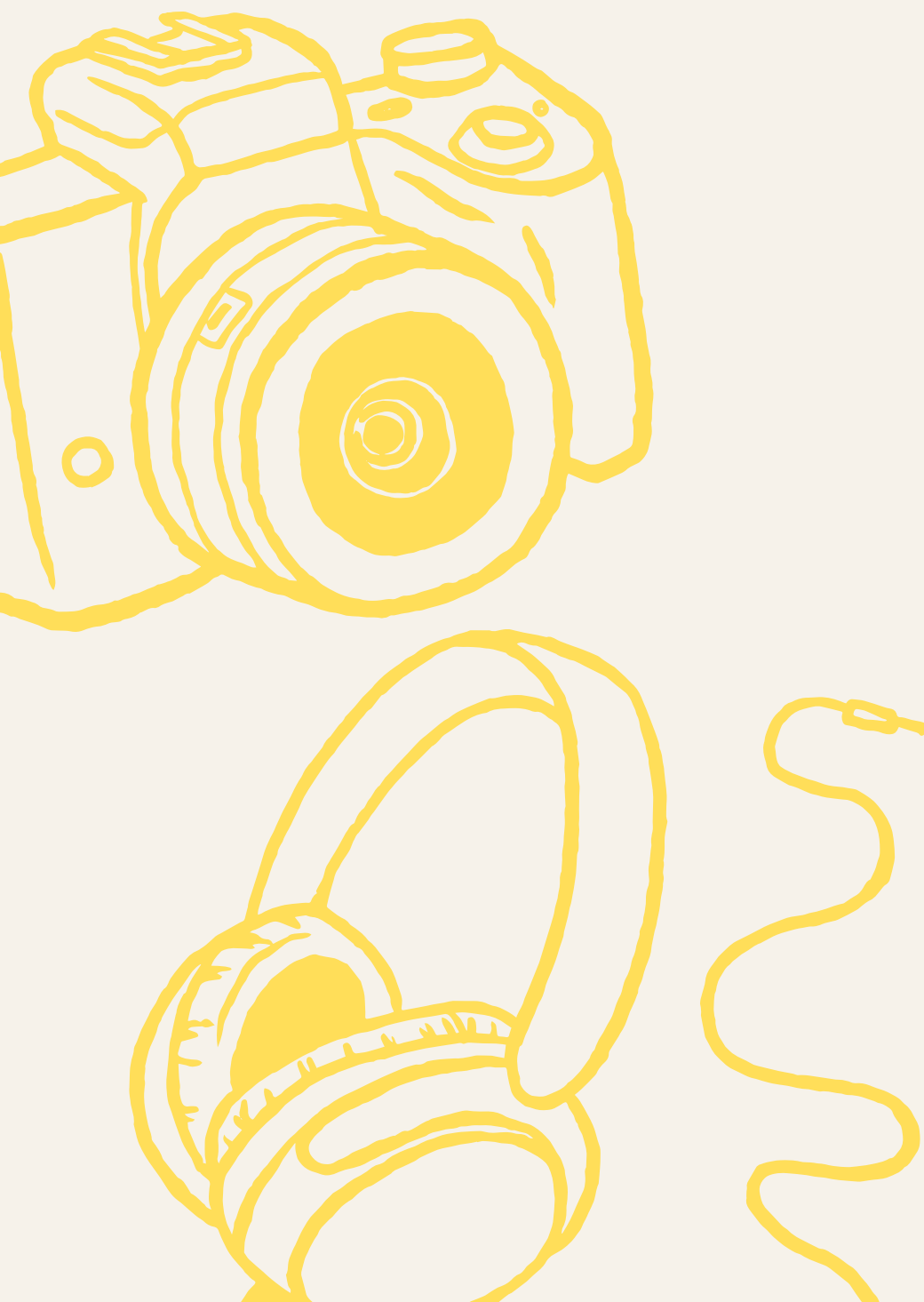
- Windows 10 或 Windows 11
- Visual Studio 2022
- Visual Studio 的「使用 C++ 的桌面開發」工作負載
- Git（使用下載 ZIP 的方式則不需要）

使用 Git 下載

git clone <https://github.com/jimmy0579/ssss.git>

使用 Visual Studio 執行

1. 開啟專案資料夾中的 ssss.sln。
2. 將組態選擇為 Debug，平台選擇為 x64。
3. 點選「建置」→「建置方案」，或按下 Ctrl + Shift + B。 9
4. 按下 Ctrl + F5 執行遊戲。



運行畫面截圖

遊戲開始畫面



遊戲結束畫面



程式畫面截圖

```
#include "SnakeGame.h"

int main() {
    Game game;
    game.run();

    return 0;
}

#ifndef SNAKEGAME_H
#define SNAKEGAME_H

#include <deque> //這裡用來存蛇的身體。
#include <vector>
#include <string>
#include <random>
#include <conio.h> //這兩個可以偵測鍵盤輸入。
#include <windows.h>

using namespace std;

enum Direction {
    UP,
    DOWN,
    LEFT,
    RIGHT
};
//它用來表示蛇目前的移動方向。

struct Point {
    int x;
    int y;

    bool equals(const Point& other) const;
};
//它用來表示座標。

class Snake {
private:
    deque<Point> body; //用 deque 儲存蛇的身體。每一個 Point 都代表蛇身上的一格。
    Direction direction; //儲存蛇目前的方向。
    bool needGrow;
```

```
bool Point::equals(const Point& other) const {
    return x == other.x && y == other.y;
}

// 建立蛇時，交由 reset() 統一設定初始狀態。
Snake::Snake(int startX, int startY) {
    reset(startX, startY);

    // 將蛇重設為長度 3，蛇頭位於指定座標，並朝右移動。
}

void Snake::reset(int startX, int startY) {
    body.clear();
    body.push_back({ startX, startY });
    body.push_back({ startX - 1, startY });
    body.push_back({ startX - 2, startY });

    direction = RIGHT;
    needGrow = false;

    // deque 的第一格永遠代表蛇頭。
Point Snake::getHead() const {
    return body.front();
}

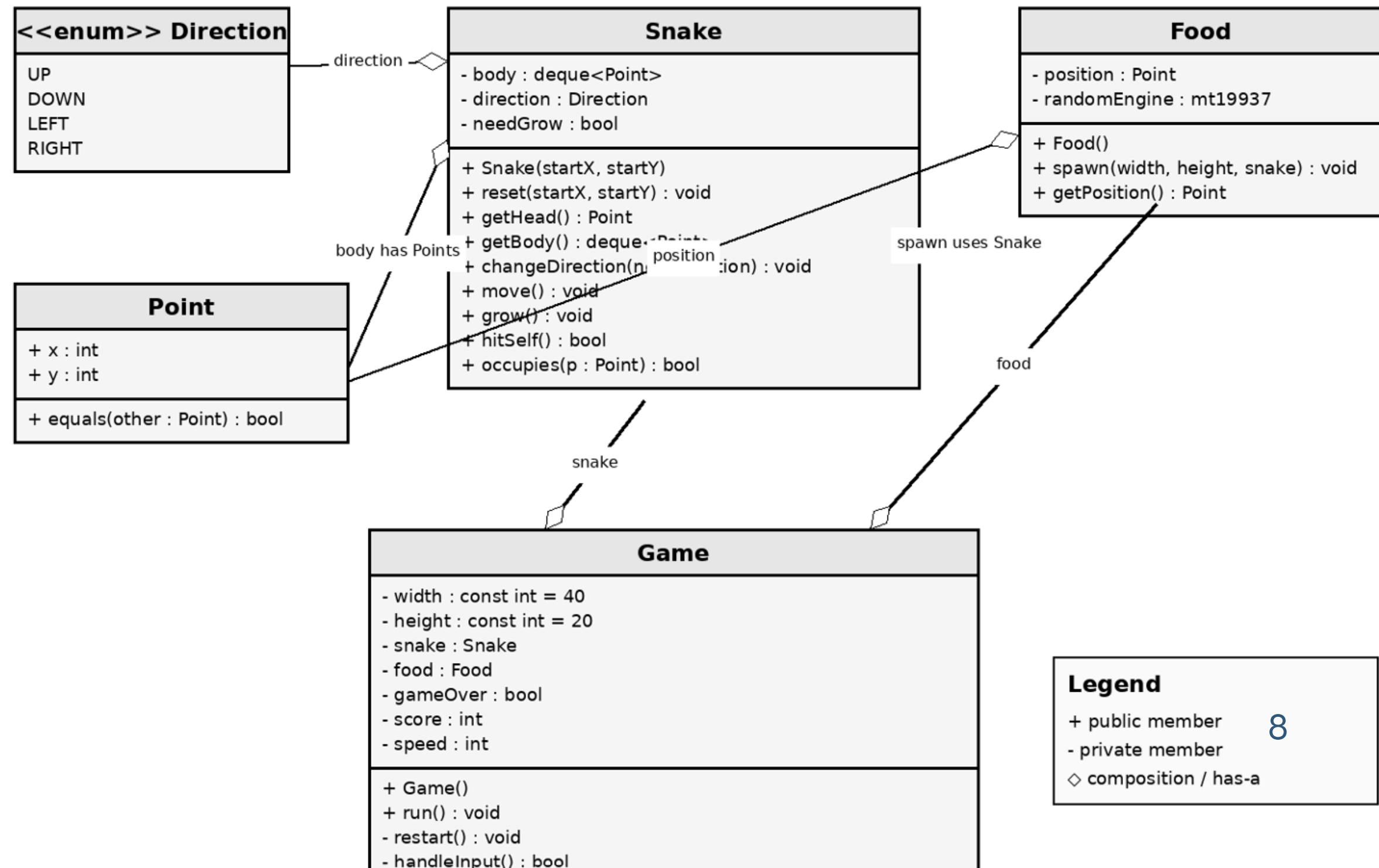
// 回傳唯讀參考，讓畫面可以讀取蛇身而不複製整個 deque。
const deque<Point>& Snake::getBody() const {
    return body;
}

void Snake::changeDirection(Direction newDirection) {
    // 禁止直接反向，避免蛇頭下一步立刻撞上第二節身體。
    if (direction == UP && newDirection == DOWN) return;
    if (direction == DOWN && newDirection == UP) return;
```

Travel Plan

UML圖

Snake Game UML Class Diagram



Travel Plan

分工

胡閔傑	主題討論、遊戲整體架構、撰寫程式碼、功能測試、製作README、製作簡報
鍾文迪	主題討論、功能測試、簡報製作
萬紫菱	主題討論、功能測試、簡報製作

github link

Bucket list //

結束